Next-Token Prediction *is* Geometry

A bridge between conditional probability and high-dimensional manifolds

One could sum up this whole document in a sentence:

> A large language model learns a map whose image is a sub-manifold of a very high-dimensional ambient space, on which geometry tracks meaning.

Arbitrary gibberish gets mapped to arbitrary points in the ambient space, but *meaningful* language gets mapped to a lower-dimensional sub-manifold.

We are using the term *manifold* loosely: the set of vectors the model produces need not be smooth, need not have constant dimension, and may be singular or stratified.

Next token prediction

A large language model is a function that takes a context and returns a probability distribution over the next token:

$$P(\text{next} \mid \text{context}).$$

That is the whole type signature, and the "statistical next-token prediction" view has it exactly right. Sampling from that distribution, one token at a time, is the entire performance.

The one equation that is the bridge

Inside the model, the context is compressed to a single vector $h \in \mathbb{R}^d$, with d a few thousand. Every candidate token w carries its own vector, its embedding e_w, living in the *same* space. The probability we want is then

$$P(w \mid \text{context}) \;=\; \operatorname{softmax}_w\langle h,\, e_w\rangle \;=\; \frac{\exp\langle h,\, e_w\rangle}{\displaystyle\sum_{w'} \exp\langle h,\, e_{w'}\rangle}.$$

The inner product $\langle h, e_w\rangle$ is nothing but *how aligned the context's direction is with the token's direction* — an angle and a length. The softmax is merely the rule that turns a list of alignments into a list of probabilities summing to one. So:

> **The conditional probability is high exactly when the context vector points the same way as the token vector.**

"Likely next word" and "nearby direction in high-dimensional space" are not two facts. They are one fact, written in two different notations.

Mathematics

A language model works with a fixed, finite list of symbols chosen before training — the *vocabulary* — whose n entries are its *tokens*. Each token w carries its embedding vector e_w, and predicting the next token means choosing one entry from this list.

The context — the tokens so far — is read by the model and compressed into a single vector h of the same dimension, so it lives in the same space as the token embeddings. It has to: the model predicts by comparing h against each token's vector and asking which it best matches, and such a comparison only means something when the two vectors share a space. So h is built precisely to be measured against the e_w — the context rendered as a point among the tokens.

Recall how the inner product of two vectors encodes the angle between them:

$$\langle u, v \rangle = \|u\| \, \|v\| \cos\theta,$$

where θ is the angle separating them.

A *logit* is the raw, un-normalised score the model gives a token before it becomes a probability:

$$z_w = \langle h, e_w \rangle.$$

The vector of all n logits is what softmax turns into probabilities.

Recall two facts about the exponential function: it is always positive, $e^x > 0$, and it is equal to its own derivative, $\frac{d}{dx} e^x = e^x$.

Given a vector of logits $z = (z_1, \ldots, z_n) \in \mathbb{R}^n$ — one entry per vocabulary token — the *softmax* is

$$\operatorname{softmax}(z)_i \;=\; \frac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}} \;=:\; p_i.$$

It is a probability distribution. Two checks. Each numerator e^{z_i} is strictly positive and the denominator is a sum of positives, so $p_i > 0$ for every i; and they normalise,

$$\sum_{i=1}^{n} p_i = \frac{\sum_{i=1}^{n} e^{z_i}}{\sum_{k=1}^{n} e^{z_k}} = 1.$$

Recall two facts about the logarithm: it is the inverse of the exponential, and it turns products into sums, $\log(ab) = \log a + \log b$. As a consequence it turns quotients into differences, $\log(a/b) = \log a - \log b$.

Now, writing $S = \sum_k e^{z_k}$,

$$\log p_w = z_w - \log S.$$

Differences of logits shed that constant and give us *log-odds*,

$$z_i - z_j = \log \frac{p_i}{p_j}.$$

Recall the quotient rule from calculus: for differentiable functions f and g,

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}.$$

On the diagonal ($i = j$), the quotient rule gives

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i} \, S - e^{z_i} \, e^{z_i}}{S^2} = p_i - p_i^2 = p_i \, (1 - p_i).$$

Off the diagonal ($i \neq j$),

$$\frac{\partial p_i}{\partial z_j} = e^{z_i} \cdot \left(-\frac{1}{S^2}\right) e^{z_j} = -p_i\, p_j.$$

The loss

Recall two additional facts about the logarithm. $\log(1) = 0$ and $\log(x)$ tends to negative infinity as x tends to 0.

Define the log loss by

$$L = -\log p_y.$$

If the model is certain that the next token is y then the loss is zero. If the model has a low probability of predicting the correct next token then the loss is large.

The corpus the model trains on is a collection of (context, next-token) pairs. The total loss is the sum of the per-pair losses,

$$L_{\text{total}} = \sum_t -\log p_{y_t} = -\log \prod_t p_{y_t},$$

The product $\prod_t p_{y_t}$ is the probability of independent events. Minimizing the total loss is the same as maximizing $\prod_t p_{y_t}$. Maximizing the probability that you will guess the correct next token over the entire corpus.

Differentiating the loss

Recall the derivative of the logarithm:

$$\frac{d}{dx} \log x = \frac{1}{x}.$$

Now, by the chain rule,

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial p_y} \frac{\partial p_y}{\partial z_j} = -\frac{1}{p_y} \frac{\partial p_y}{\partial z_j}.$$

Now use the derivatives of the softmax that we calculated earlier. For $j = y$,

$$\frac{\partial L}{\partial z_j} = -\frac{1}{p_y}\, p_y(1 - p_y) = -(1 - p_y) = p_y - 1 = p_j - 1.$$

For $j \neq y$,

$$\frac{\partial L}{\partial z_j} = -\frac{1}{p_y}\, (-p_y p_j) = p_j.$$

We combine these two facts using the Kronecker delta notation:

$$\frac{\partial L}{\partial z_j} = p_j - \delta_{jy}.$$

The gradient is a spring law

Recall that the logits are defined by the inner product $z_w = \langle h, e_w \rangle$.

The embedding of the context h is a vector, so we expand it in the standard basis,

$$h = \sum_i h_i \, \hat{e}_i.$$

Written in coordinates, each logit is a dot product: expanding h in the basis,

$$z_w = \langle h, e_w \rangle = \left\langle \sum_i h_i \, \hat{e}_i, \; e_w \right\rangle = \sum_i h_i \, \langle \hat{e}_i, e_w \rangle.$$

So z_w is a linear function of the h_i.

The loss is a function of these logits, so the chain rule gives

$$\frac{\partial L}{\partial h_i} = \sum_w \frac{\partial L}{\partial z_w} \frac{\partial z_w}{\partial h_i} = \sum_w \frac{\partial L}{\partial z_w} \langle \hat{e}_i, e_w \rangle.$$

Stacking the coordinates back into a vector,

$$\frac{\partial L}{\partial h} = \sum_i \frac{\partial L}{\partial h_i} \hat{e}_i,$$

and swapping the order of the two sums,

$$\frac{\partial L}{\partial h} = \sum_i \sum_w \frac{\partial L}{\partial z_w} \langle \hat{e}_i, e_w \rangle \, \hat{e}_i = \sum_w \frac{\partial L}{\partial z_w} \Big(\sum_i \langle \hat{e}_i, e_w \rangle \, \hat{e}_i \Big) = \sum_w \frac{\partial L}{\partial z_w} \, e_w.$$

So, with $\partial L / \partial z_w = p_w - \delta_{wy}$,

$$\frac{\partial L}{\partial h} = \sum_w (p_w - \delta_{wy}) \, e_w = \bar{e} - e_y, \qquad \bar{e} = \sum_w p_w \, e_w.$$

Similarly, expand

$$e_\omega = \sum_i \langle e_\omega, \hat{e}_i \rangle \, \hat{e}_i.$$

so

$$\frac{\partial L}{\partial e_\omega} = \sum_i \frac{\partial L}{\partial \langle e_\omega, \hat{e}_i \rangle} \hat{e}_i.$$

Recall that the loss function is a function of the logits z_k. By the chain rule:

$$\frac{\partial L}{\partial \langle e_\omega, \hat{e}_i \rangle} = \sum_k \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial \langle e_\omega, \hat{e}_i \rangle}.$$

Since z_k depends only on e_k, if $k \neq \omega$ then varying the i-th component of e_ω does not change z_k, so

$$\frac{\partial z_k}{\partial \langle e_\omega, \hat{e}_i \rangle} = 0;$$

only the $k = \omega$ term survives:

$$\frac{\partial L}{\partial \langle e_\omega, \hat{e}_i \rangle} = \frac{\partial L}{\partial z_\omega} \frac{\partial z_\omega}{\partial \langle e_\omega, \hat{e}_i \rangle} = \frac{\partial L}{\partial z_\omega} \, h_i.$$

Substituting,

$$\frac{\partial L}{\partial e_\omega} = \sum_i \frac{\partial L}{\partial z_\omega} \, h_i \, \hat{e}_i = \frac{\partial L}{\partial z_\omega} \sum_i h_i \, \hat{e}_i = \frac{\partial L}{\partial z_\omega} \, h = (p_\omega - \delta_{\omega y}) \, h.$$

Take one training step. A context produces the hidden vector h, the true next token is y, and the model's current distribution is $p_w = \mathrm{softmax}\langle h, e_w \rangle$. The loss is $L = -\log p_y$. Differentiating is the whole mechanism in two lines:

$$\frac{\partial L}{\partial h} = \sum_w (p_w - \delta_{wy}) e_w = \bar{e} - e_y, \qquad \frac{\partial L}{\partial e_w} = (p_w - \delta_{wy})\, h, \qquad \bar{e} = \sum_w p_w\, e_w.$$

Training moves against the gradient, so the update is its negative. It says four things — two about the context, two about the embeddings.

- **The context moves toward the true token.** The update to h is $-\partial L/\partial h = e_y - \bar{e}$; its $+e_y$ part pulls h toward the embedding of the token that actually came next.
- **The context moves away from what it expected.** The same update's $-\bar{e}$ part pushes h away from $\bar{e}$, the average embedding under its current distribution.
- **The true token's embedding moves toward the context.** The update to e_y is $(1 - p_y)\, h$, pulling e_y toward h.
- **Every other token embedding moves away from the context.** The update to e_w $(w \neq y)$ is $-p_w\, h$, pushing it away from h by exactly p_w — the probability wrongly placed on it.

That is a force-directed layout. Every occurrence in the corpus is one tug: this context attracts this token's direction and repels the others. Summed over billions of examples, the attractions and repulsions reach an equilibrium — and **the equilibrium configuration is the manifold**.

Why the equilibrium is a *meaning* manifold

The output is a smooth, fixed function of h alone. So the only way the model can give two contexts similar next-token distributions is to map them to nearby h. Contexts followed by similar things are forced together; contexts followed by different things are forced apart. That is the distributional hypothesis — *you shall know a word by the company it keeps* — made mechanical.